
DOCUMENT DE SPÉCIFICATIONS FONCTIONNELLES

Projet Web Dynamique 2026 — Sujet 2 : SmartCampus

Auteurs :

Viktor	GOUSSOT
Nicolas	CARMINATI
Alexis	OSORIO
Louis	PEZE

Enseignant :

M. HINA

RÉSUMÉ — SmartCampus est une plateforme web dynamique de gestion académique destinée à une école d'ingénieurs. Elle centralise les interactions entre l'administration, les enseignants et les étudiants autour des cours, des inscriptions, des notes et des emplois du temps. Trois rôles utilisateurs (administrateur, enseignant, étudiant) accèdent à des fonctionnalités cloisonnées par un contrôle d'accès serveur (RBAC). Le projet répond au sujet « Projet Web dynamique 2026 — Sujet 2 » en respectant les contraintes imposées : base de données relationnelle (MySQL), backend PHP organisé en API REST, frontend HTML/CSS/JavaScript avec Bootstrap, et la mise en œuvre de plusieurs règles métier réalistes (capacité, prérequis, conflits d'emploi du temps, verrouillage des notes). Ce document présente les rôles utilisateurs, les fonctionnalités principales, les contraintes techniques, les technologies envisagées et la répartition prévisionnelle du travail entre les membres de l'équipe.

Table des matières

I. Objectif du document	3
II. Glossaire	3
A. Termes	3
B. Acronymes	3
III. L'équipe	4
A. Présentation de l'équipe	4
B. Organisation de l'équipe	4
C. Répartition prévisionnelle du travail	4
IV. Contexte et problématique	5
A. Contexte	5
B. Problématique	5
C. Public cible	5
V. Rôles utilisateurs	6
A. Visiteur	6
B. Étudiant	6
C. Enseignant	6
D. Administrateur	6
VI. Principales fonctionnalités	7
A. Tableau récapitulatif	7
B. Règles métier	7
C. Cas d'utilisation principal	8
VII. Contraintes techniques et technologies envisagées	9
A. Contraintes techniques imposées	9
B. Technologies retenues	9
C. Architecture envisagée	10
VIII. Interfaces attendues	11
IX. Critères de validation	11
X. Bibliographie	12

I. Objectif du document

Ce document présente les **spécifications fonctionnelles** du projet SmartCampus, plateforme web dynamique de gestion académique développée dans le cadre du module « Projet Web dynamique 2026 — Sujet 2 ». Il s'adresse aux encadrants pédagogiques et constitue la première livraison du projet (livrable conception, dû le **28 mai 2026**).

Le lecteur y trouvera :

- la description des **rôles utilisateurs** et de leurs permissions ;
- les **fonctionnalités principales** envisagées (cœur du sujet et bonus) ;
- les **règles métier** qui sécurisent les inscriptions, les notes et l'emploi du temps ;
- les **contraintes techniques** imposées par le sujet et les **technologies retenues** ;
- une **première répartition prévisionnelle du travail** entre les membres de l'équipe.

II. Glossaire

A. Termes

Terme	Définition
Filière	Spécialité d'études choisie par l'étudiant (informatique, mécanique, énergie, etc.).
Niveau	Année académique : L1, L2, L3 (licence) puis M1, M2 (master).
Semestre	Période d'enseignement (S1 à S10), associée à un niveau.
Groupe de TD	Sous-groupe d'étudiants pour les travaux dirigés.
Prérequis	Cours qu'un étudiant doit avoir validé (moyenne ≥ 10) pour s'inscrire à un autre cours.
Verrouillage d'une note	Action de l'enseignant rendant une note définitive et non modifiable.
Règle métier	Contrainte applicative qui traduit une règle de gestion réelle (ex. capacité d'un cours).
Catalogue	Vue d'ensemble des cours proposés, filtrable par niveau, semestre, département.

B. Acronymes

Acronyme	Signification	Explication
API	Application Programming Interface	Interface permettant au frontend de dialoguer avec le serveur via HTTP/JSON.
REST	Representational State Transfer	Style d'architecture d'API basé sur les méthodes HTTP (GET, POST, PUT, DELETE).
SPA	Single Page Application	Application web rendue côté client à partir d'une seule page HTML.
MVC	Modèle – Vue – Contrôleur	Patron d'architecture séparant données, affichage et logique.
RBAC	Role-Based Access Control	Contrôle d'accès fondé sur les rôles utilisateurs.
CSRF	Cross-Site Request Forgery	Attaque par requête forgée ; contrée par un jeton secret par session.
PDO	PHP Data Objects	Couche d'abstraction de PHP pour accéder aux bases de données (requêtes préparées).
ECTS	European Credits Transfer System	Système européen de crédits universitaires.
CRUD	Create, Read, Update, Delete	Les quatre opérations de base sur une ressource.
MCD	Modèle Conceptuel de Données	Représentation abstraite des entités et associations du domaine.

III. L'équipe

A. Présentation de l'équipe

#	Nom / Prénom	Compétences principales
1	Viktor GOUSSOT	Chef de projet — Backend (PHP, sécurité, architecture API)
2	Nicolas CARMINATI	Backend (règles métier, base de données, requêtes SQL)
3	Alexis OSORIO	Frontend (HTML/CSS, Bootstrap, ergonomie, intégration)
4	Louis PEZE	Frontend (JavaScript, dashboards, tests)

B. Organisation de l'équipe

- **Versioning** : dépôt Git/GitHub partagé, commits réguliers et nommés avec préfixes (`feat:` , `fix:` , `docs:` ...). Branches par fonctionnalité fusionnées sur `main` par *pull request* après relecture croisée.
- **Communication** : points d'avancement hebdomadaires ; chaque membre doit pouvoir expliquer le code des autres (objectif pédagogique du module).
- **Méthode** : approche itérative — d'abord le squelette (authentification, base de données, routes CRUD), puis les règles métier, puis les écrans, puis les bonus (PDF, messagerie, statistiques).

C. Répartition prévisionnelle du travail

Module / Tâche	Responsable	Contributeurs
Modèle de données (MCD, schéma SQL)	Nicolas CARMINATI	Viktor GOUSSOT
Noyau backend (Router, Auth, sécurité)	Viktor GOUSSOT	—
Règles métier (inscriptions, notes, conflits)	Nicolas CARMINATI	Viktor GOUSSOT
API CRUD (étudiants, enseignants, cours)	Viktor GOUSSOT	Nicolas CARMINATI
Authentification & gestion de session	Viktor GOUSSOT	—
Frontend — noyau (routeur, UI, services)	Alexis OSORIO	Louis PEZE
Frontend — pages de gestion (CRUD)	Alexis OSORIO	—
Frontend — espace étudiant (notes, inscriptions)	Louis PEZE	Alexis OSORIO
Tableaux de bord par rôle	Louis PEZE	—
Emploi du temps (UI + API)	Louis PEZE	Nicolas CARMINATI
Bonus — relevé PDF (FPDF)	Nicolas CARMINATI	—
Bonus — messagerie & notifications	Alexis OSORIO	Louis PEZE
Bonus — statistiques (Chart.js)	Louis PEZE	—
Documentation & rapport	Tous	—
Tests & démonstration	Louis PEZE	Tous

Cette répartition est *prévisionnelle* : elle sera ajustée en cours de projet selon l'avancement et la charge réelle, et la version définitive sera présentée lors de la soutenance avec les commits Git à l'appui.

IV. Contexte et problématique

A. Contexte

Les établissements d'enseignement supérieur gèrent un volume croissant d'informations académiques : inscriptions aux cours, suivi des notes, planification des cours, mais aussi communication entre l'administration, les enseignants et les étudiants. Ces informations sont aujourd'hui souvent dispersées entre plusieurs outils (intranet, tableurs, courriels), ce qui complique le suivi pédagogique et la prise de décision.

Le présent projet s'inscrit dans le module « Projet Web dynamique 2026 » de l'ECE. Le sujet n° 2 demande la conception d'une plateforme web dynamique de gestion académique. Nous avons choisi comme contexte une **école d'ingénieurs** (organisation en filières, niveaux L1 → M2, semestres, crédits ECTS, groupes de TD).

B. Problématique

Comment concevoir une plateforme web unique, ergonomique et sécurisée, qui centralise la gestion académique d'une école d'ingénieurs en garantissant à chaque rôle utilisateur un accès strictement délimité à ses fonctionnalités, tout en appliquant des règles métier réalistes (capacité, prérequis, conflits horaires, verrouillage des notes) ?

Cette problématique se décline en plusieurs sous-questions :

- Quel modèle de données permet de représenter à la fois les étudiants, les enseignants, les cours, les inscriptions, les notes et l'emploi du temps sans duplication d'information ?
- Comment cloisonner les droits de chaque rôle sans dépendre uniquement de l'interface graphique (sécurité côté serveur) ?
- Comment garantir la cohérence des données face à des actions concurrentes (deux étudiants qui s'inscrivent en même temps à la dernière place d'un cours, par exemple) ?

C. Public cible

Public	Besoin principal
Étudiants	Consulter rapidement leurs cours, leurs notes et leur emploi du temps ; s'inscrire ou se désinscrire d'un cours ; télécharger un relevé de notes.
Enseignants	Voir les étudiants inscrits à leurs cours, saisir et valider les notes, consulter leur emploi du temps.
Administration pédagogique	Gérer le référentiel (étudiants, enseignants, cours), planifier l'emploi du temps, suivre la réussite via des statistiques.

V. Rôles utilisateurs

La plateforme distingue quatre rôles. Le cloisonnement est **imposé côté serveur** (RBAC) — il ne dépend pas seulement de l'interface affichée.

A. Visiteur

- Consulter la page d'accueil et de connexion.
- Créer un compte étudiant (inscription publique).
- Aucun accès aux données académiques.

B. Étudiant

- Consulter le catalogue des cours et **s'inscrire / se désinscrire** (sous réserve des règles métier).
- Consulter ses propres **notes** et calculs de moyenne, télécharger un **relevé PDF**.
- Consulter son **emploi du temps** hebdomadaire.
- Utiliser la **messagerie interne** avec ses enseignants.
- Recevoir des **notifications** (note publiée, inscription validée...).

C. Enseignant

- Consulter la liste de **ses** cours et des étudiants inscrits.
- **Saisir, modifier et valider** les notes (chaque note peut être verrouillée — elle devient alors définitive).
- Consulter son emploi du temps.
- Échanger des messages avec ses étudiants.
- *Restriction* : un enseignant ne peut noter que les étudiants des cours qu'il encadre.

D. Administrateur

- Gérer le référentiel : **étudiants, enseignants, cours, créneaux** (CRUD complet).
- **Modérer** les utilisateurs (activer / désactiver / supprimer).
- Inscrire ou désinscrire un étudiant d'un cours en cas de besoin administratif.
- Consulter des **statistiques** de réussite et de répartition.
- Aucune restriction de visibilité (vue globale).

VI. Principales fonctionnalités

A. Tableau récapitulatif

ID	Fonctionnalité	Description	Rôles
F01	Authentification	Connexion / déconnexion / inscription, sessions sécurisées.	Tous
F02	Gestion des utilisateurs	Création de comptes, gestion des rôles, modération (activer / désactiver / supprimer).	Admin
F03	Gestion des étudiants	CRUD étudiant, profil, parcours académique, recherche / filtre / tri.	Admin, Enseignant (lecture)
F04	Gestion des enseignants	CRUD enseignant, association aux cours.	Admin
F05	Gestion des cours	CRUD cours, affectation enseignant, archivage, filtres / tri.	Admin
F06	Inscriptions aux cours	Inscription / désinscription, listes des inscrits.	Étudiant, Admin
F07	Gestion des notes	Saisie, modification, calcul de moyenne, validation finale (verrouillage).	Enseignant, Admin
F08	Emploi du temps	Création de créneaux, consultation hebdomadaire selon le rôle.	Admin (gestion), tous (lecture)
F09	Tableaux de bord	Vue synthétique adaptée au rôle (statistiques, prochaines séances, notes récentes).	Tous
F10	Relevé de notes PDF (bonus)	Génération PDF du relevé d'un étudiant.	Étudiant, Admin, Enseignant
F11	Messagerie interne (bonus)	Échange de messages entre utilisateurs selon les rôles.	Tous
F12	Notifications (bonus)	Alertes : note publiée, inscription, validation, message reçu.	Tous
F13	Statistiques (bonus)	Graphiques académiques (réussite, répartitions par filière / niveau).	Admin

B. Règles métier

Le sujet exige plusieurs règles métier réalistes. **Six** règles sont prévues, toutes vérifiées **côté serveur** :

1. **Pas de double inscription** — un étudiant ne peut s'inscrire deux fois au même cours.
2. **Contrôle de capacité** — l'inscription est refusée si la capacité maximale du cours est atteinte.
3. **Prérequis académiques** — l'inscription à un cours exige d'avoir validé (moyenne ≥ 10) les cours prérequis.

4. **Détection des conflits d'emploi du temps** — un étudiant ne peut suivre deux cours qui se chevauchent ; deux créneaux ne peuvent pas être placés dans la même salle ou pour le même enseignant au même moment.
5. **Verrouillage des notes après validation** — une note validée devient non modifiable et non supprimable.
6. **Restriction des actions selon les rôles** — chaque action sensible vérifie le rôle et, pour l'enseignant, la propriété du cours.

C. Cas d'utilisation principal — « S'inscrire à un cours »

- **Acteur principal** : étudiant connecté.
- **Préconditions** : l'étudiant est authentifié ; le cours existe et n'est pas archivé.
- **Scénario nominal** :
 1. L'étudiant recherche / filtre un cours dans le catalogue.
 2. Il clique sur « S'inscrire ».
 3. Le serveur vérifie successivement : double inscription, capacité, prérequis, conflit d'emploi du temps.
 4. Si toutes les règles passent, l'inscription est enregistrée et une notification est créée.
- **Scénarios alternatifs** : à chaque règle non respectée, l'inscription est refusée avec un message explicite (ex. « Ce cours est complet », « Prérequis non validés : INFO-203 », « Conflit d'emploi du temps avec MATH-202 »).

VII. Contraintes techniques et technologies envisagées

A. Contraintes techniques imposées par le sujet

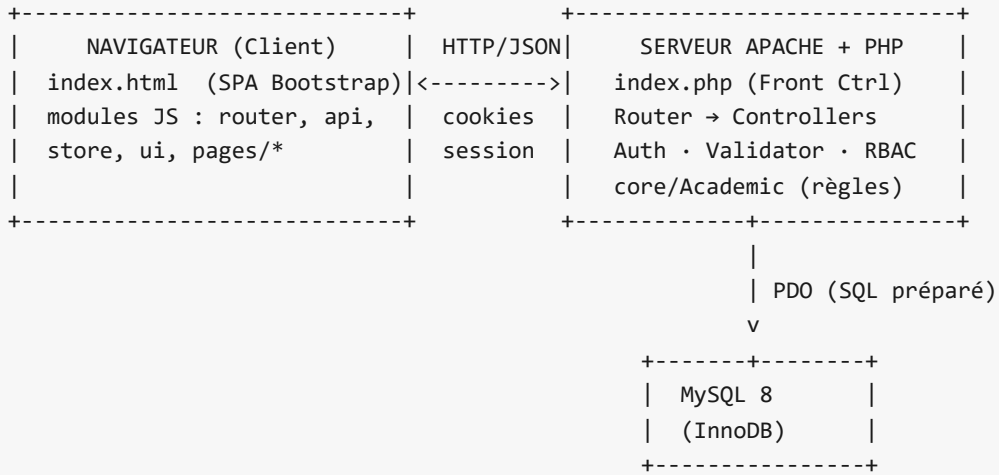
- Base de données **relationnelle** (MySQL ou équivalent).
- Backend en **PHP** organisé en API REST, séparé du frontend.
- Frontend en **HTML / CSS / JavaScript**, avec Bootstrap pour la mise en forme.
- Validation des entrées **côté client ET côté serveur**.
- Sécurité : sessions, jeton anti-CSRF, contrôle d'accès par rôle, requêtes **préparées** (anti-injection SQL), **hashage** des mots de passe.
- Interface **responsive** (testée mobile et desktop).
- Pas de framework lourd type Laravel ou Symfony (architecture maîtrisée, explicable).

B. Technologies retenues

Couche	Technologie	Justification
Base de données	MySQL 8 (moteur InnoDB)	Imposé par le sujet. InnoDB pour les clés étrangères et les transactions (cohérence lors des actions multi-tables).
Backend	PHP 8 (sans framework, mini-MVC maison)	Imposé par le sujet. Un mini-noyau (≈ 6 classes) rend chaque mécanisme explicable en soutenance.
Accès BDD	PDO (requêtes préparées)	Standard PHP ; protège contre les injections SQL.
Frontend	HTML5, CSS3, JavaScript ES modules	Sans build (pas de Webpack / Vite) : modules ES natifs chargés dynamiquement.
UI	Bootstrap 5 + Bootstrap Icons	Imposé par le sujet ; accélère la mise en forme et garantit le responsive.
Graphiques	Chart.js	Léger, simple, sans dépendance ; suffisant pour les statistiques de l'admin.
Génération PDF	FPDF (PHP)	Bibliothèque éprouvée pour produire des relevés de notes côté serveur.
Serveur de dev	WAMP (Apache + PHP + MySQL) sous Windows	Environnement local standard pour les projets PHP à l'école.
Versioning	Git + GitHub	Suivi des contributions, branches par fonctionnalité, relecture croisée.

C. Architecture envisagée

Architecture **client – serveur** avec séparation nette frontend / backend. Le frontend (Single Page Application) appelle l'API REST en JSON ; l'API authentifie, vérifie les droits (RBAC) et applique les règles métier avant d'écrire en base.



VIII. Interfaces attendues

Page	Contenu
Connexion / Inscription	Formulaires d'authentification (e-mail + mot de passe).
Tableau de bord	Synthèse adaptée au rôle : statistiques, prochaines séances, notes récentes.
Étudiants / Enseignants	Listes filtrables + formulaires CRUD en modale.
Catalogue des cours	Cartes filtrables / triables, inscription, gestion.
Détail d'un cours	Informations, créneaux, grille de notes (enseignant), inscription (étudiant).
Mes notes	Notes par cours, moyennes, résultat, téléchargement du relevé PDF.
Emploi du temps	Vue hebdomadaire lundi → vendredi.
Messagerie	Boîtes reçue / envoyée + composition.
Statistiques	Graphiques (Chart.js) — réservé à l'administrateur.
Utilisateurs	Modération des comptes — réservé à l'administrateur.

IX. Critères de validation

Les critères de validation correspondent aux exigences du sujet :

- Inscription et authentification fonctionnelles.
- Processus d'inscription aux cours complet, avec règles métier.
- Ajout / retrait de cours et de données académiques par l'administration.
- Modération de la plateforme par l'administrateur.
- Interface claire, ergonomique et responsive.
- Calcul automatique des moyennes et détermination du résultat (admis / non admis).
- Sécurité : sessions, CSRF, RBAC, requêtes préparées, hashage des mots de passe.

X. Bibliographie

1. PHP Group, « PHP Manual », [En ligne]. Disponible : <https://www.php.net/manual/fr/>. [Accès le 28 mai 2026].
2. Oracle, « MySQL 8.0 Reference Manual », [En ligne]. Disponible : <https://dev.mysql.com/doc/refman/8.0/en/>. [Accès le 28 mai 2026].
3. Bootstrap Team, « Bootstrap 5 documentation », [En ligne]. Disponible : <https://getbootstrap.com/docs/5.3/>. [Accès le 28 mai 2026].
4. Mozilla Developer Network, « JavaScript — Guide », [En ligne]. Disponible : <https://developer.mozilla.org/fr/docs/Web/JavaScript>. [Accès le 28 mai 2026].
5. OWASP Foundation, « OWASP Top 10:2021 », [En ligne]. Disponible : <https://owasp.org/Top10/>. [Accès le 28 mai 2026].
6. Chart.js Contributors, « Chart.js documentation », [En ligne]. Disponible : <https://www.chartjs.org/docs/latest/>. [Accès le 28 mai 2026].
7. O. Plathey, « FPDF Library », [En ligne]. Disponible : <http://www.fpdf.org/>. [Accès le 28 mai 2026].